KruppAI Development Summary

Sprint 1 Complete

Date: February 14, 2026 | Test Suite: 311/311 passing | Branch: claude/skill-development-plan-85oL0

Commit History

Hash	Message
dce6ba5	KruppAI: Complete architecture, build prompts, schema, and development plan
4f88e86	Foundation build: complete core framework, CLI, tests (78/78 passing)
58e83c7	Phase 1 skills: 6 working CLI commands with full test suite (177/177 passing)
edd205e	Phase 2 skills: 6 document analysis skills with full test suite (237/237 passing)
f8311df	Phase 3 complete: 7 skills, web UI, FastAPI backend, Render deployment (311/311 passing)
3e5df9a	Premium UI overhaul: project-first flow, rich skill details, better error handling
50bca8d	Add UI redesign plan based on first-principles review
d66629a	Implement Project Command Center redesign (PLAN.md)

18 AI Document Generation Skills

#	Skill	Ph	Model	Output	Description
1	Daily Field Report	1	Sonnet	DOCX	Rough field notes to professional daily reports with auto-fetched weather
2	RFI Generator	1	Sonnet	DOCX	Issue descriptions to formal RFI documents with auto-numbering
3	Meeting Minutes	1	Sonnet	DOCX	Rough notes to formatted minutes with attendee & action item tracking
4	Client Update Letter	1	Sonnet	DOCX	Professional project status letters on Krupp letterhead
5	Toolbox Safety Talk	1	Sonnet	DOCX	5-10 minute safety talks with OSHA references
6	Punch List Generator	1	Sonnet	DOCX+XLSX	Walk-through notes to trade-organized punch lists
7	Estimate Reviewer	2	Opus	DOCX+XLSX	Cost estimates reviewed against industry benchmarks by CSI division
8	Bid Comparison	2	Sonnet	DOCX+XLSX	Multiple bids compared side-by-side with best-value recommendation
9	Change Order Builder	2	Sonnet	DOCX	Formal change order proposals with cost breakdown & markup
10	Schedule Variance	2	Sonnet	DOCX+XLSX	Schedule analysis for variance, critical path, at-risk activities
11	Submittal Tracker	2	Sonnet	DOCX+XLSX	Submittal logs analyzed, overdue items flagged
12	Contract Checker	2	Opus	DOCX	Contracts/insurance reviewed for compliance & non-standard clauses
13	Proposal Generator	3	Opus	DOCX	Full proposals, letter proposals, or SOQs from knowledge base
14	Budget Forecaster	3	Sonnet	DOCX+XLSX	Job cost analysis, final cost projection, risk identification
15	Closeout Assembler	3	Sonnet	DOCX+XLSX	Closeout package with branded cover letter & tracker
16	Lessons Learned	3	Sonnet	DOCX	Structured lessons in manual or extract (data mining) modes
17	Case Study Generator	3	Sonnet	DOCX	Polished marketing case studies, audience-tailored
18	Incident Report	3	Sonnet	DOCX	Formal incident reports with root cause & OSHA classification

Every skill follows a **BaseSkill** contract: validate_input → build_prompt → format_output, producing branded DOCX/XLSX with Krupp branding (navy #1B3A5C, gold #D4A84B, Calibri font).

Core Framework (12 Modules)

Module	Purpose
config.py	Pydantic settings loader with .env file support and KRUPPAI_ prefix
database.py	SQLite connection manager with auto-schema initialization
api_client.py	Anthropic API wrapper with retry logic (3x exponential backoff), cost tracking
document_parser.py	Universal parser for PDF, DOCX, XLSX, images, and plain text
output_formatter.py	Branded DOCX/XLSX generation with consistent naming convention
context_manager.py	Loads full project context (team, subs, open items) for prompt assembly
knowledge_base.py	Injects company profile, writing standards, safety standards into prompts
weather.py	Auto-fetches current conditions from Open-Meteo (free, no API key)
security.py	Input sanitization, path validation, prompt injection defense
monitoring.py	Health checks, cost tracking summaries
migrate_to_supabase.py	SQLite to Supabase PostgreSQL migration utility

Database Schema

23 tables covering projects, team members, subcontractors, and all 18 skill-specific data stores (daily_reports, rfis, meetings, action_items, change_orders, punch_lists, etc.) plus system tables for API usage tracking and document management.

4 views: v_active_projects (with computed open RFI/action item counts), v_open_action_items, v_api_cost_summary, v_subcontractor_performance.

Two Interfaces

- **CLI** (Click + Rich) — 20+ commands including interactive 18-skill guided menu.
- **Web** (FastAPI) — REST API with vanilla HTML/CSS/JS SPA. Endpoints for health, config, project CRUD, skill execution, document download, project stats, activity feed.

Deployment

- **Render** — render.yaml blueprint for one-click deploy (Docker, starter plan, 1GB persistent disk)
- **Supabase** — PostgreSQL schema ready with migration utility
- **Docker** — Dockerfile included for containerized deployment

UI Redesign: Project Command Center

The initial UI was redesigned based on first-principles user feedback. The original interface presented skills as a flat list with modal popups — functionally correct but indistinguishable from "just using Claude directly."

What Changed

Before	After
Welcome hero with 4-step onboarding	Direct "Create Project" form if no projects exist
Cost metrics front-and-center	Cost tracking moved to Settings screen
Modal popups for every skill	Inline form expansion within workflow groups
Skills listed by phase number	Skills organized by construction workflow
Static skill descriptions	Live project data in each workflow group
Flat document table	Dual tabs: My Documents + Project Record

Dashboard States

1. No projects — Dashboard IS the create-project form. No fluff.

2. Projects exist, none selected — Project grid with "Select a project to get started."

3. Project selected — Full command center: status bar, workflow groups, activity feed.

Workflow Groups

Skills organized by how construction professionals actually work:

- **Field Work** — Daily Report, Safety Talk, Incident Report
- **Design Coordination** — RFI, Submittal Tracker
- **Financial Management** — Change Order, Budget Forecast, Bid Comparison, Estimate Reviewer
- **Communication** — Client Update, Meeting Minutes
- **Project Closeout** — Punch List, Closeout, Lessons Learned, Case Study
- **Business Development** — Proposal Generator, Contract Checker

Auto-Fill Preview

Every inline form shows a preview of what the system auto-fills — weather data, auto-numbered sequence, project context counts, company standards. This is the key differentiator: the user sees exactly what work the system does that they'd have to do manually in Claude.

New API Endpoints

- **GET /api/v1/projects/{code}/stats** — Open RFIs, pending COs, action items, punch items, daily report count, auto-numbers
- **GET /api/v1/projects/{code}/activity** — Merged activity feed across documents, RFIs, COs, action items
- **GET /api/v1/documents/all** — All documents across all projects ("My Documents")

Key Insights from Development

1. The value isn't in the AI generation — it's in the PROJECT MEMORY

The database layer is the real differentiator. Every document generated feeds structured data back into queryable tables (RFI counts, change order totals, action items, punch items). Over time, the system knows the project's state in a way that a raw Claude conversation never could.

2. Make the automation visible

The original UI hid all the work the system does automatically — fetching weather, auto-numbering documents, loading project context, injecting company standards. The redesigned auto-fill preview box shows the user exactly what they'd have to

do manually.

3. **Organize by workflow, not by feature list**
   Presenting 18 skills as a numbered list was overwhelming and abstract. Grouping them into 6 construction workflows with live project data makes the tool feel like a project command center, not a document factory.

4. **Kill modals — inline everything**
   Modals break flow and feel like interruptions. Inline form expansion — click a skill, form slides open right where you clicked, shows auto-fill preview, accepts input, shows results — all in the same page flow.

5. **Cost tracking is plumbing, not a feature**
   API usage and spend belong in a Settings screen, not front-and-center on the dashboard. Users care about their project state (open RFIs, pending COs, action items) — not token consumption.

6. **Commit after each phase, not in batches**
   Context compaction during long development sessions caused uncommitted work to be lost. Lesson: push after completing each logical unit of work.

Next Sprint Recommendations

1. **Real Document Upload + OCR Pipeline**
   The DocumentParser supports PDF/DOCX/XLSX/image extraction, but the web UI file upload path needs end-to-end testing with real construction documents. Priority: superintendent field photos to daily report seamlessly.

2. **Cross-Skill Data Connections**
   The database has the data, but the UI doesn't surface relationships yet. When a user opens the Change Order form, show "Related RFIs: #3, #5" from the database. When viewing Meeting Minutes, link to action items it created.

3. **Procore / Microsoft Integrations**
   Stub modules exist in integrations/. Even a read-only Procore sync (pull project data, RFI status, submittals) would eliminate double-entry and make KruppAI the intelligence layer on top of existing tools.

4. **Multi-User + Authentication**
   Currently single-user. The Supabase migration utility and PostgreSQL schema are ready. Adding auth enables real "My Documents" with user identity, team-based project access, and audit trails.

5. **Template-Based Document Branding**
   OutputFormatter builds DOCX from scratch each time. Loading from a .docx template would let Krupp customize letterhead, headers, and formatting without code changes.

6. **Smart Defaults and Repeat Workflows**
   If someone generates a Daily Report every morning, pre-populate yesterday's crew sizes. If the same 8 people attend every OAC meeting, remember the attendee list. Reduce input to the absolute minimum.

7. **Dashboard Analytics**
   The v_api_cost_summary and v_active_projects views have the data for cost-per-document-type over time, project activity trends, and skill usage frequency. Charts in Settings would help justify ROI.

8. **Async Weather for Web API**
   fetch_weather_sync blocks the event loop in the FastAPI context. Switching to the async variant would improve responsiveness under concurrent users.

Technical Reference

File Naming Convention

```
{skill}_{project_code}_{date}_{sequence}.{ext}
```

Model Routing

- **Default:** claude-sonnet-4-5-20250929 — All Phase 1 skills, speed-critical tasks
- **Opus:** claude-opus-4-6 — Estimate Reviewer, Contract Checker, Proposal Generator

Branding

- Navy: #1B3A5C | Gold: #D4A84B | Font: Calibri
- Footer: "Generated by KruppAI | {date} | Review required before distribution"

Test Suite

311 tests across 38 files: 7 core module tests, 18 skill tests (one per skill), 1 CLI test, 3 integration tests (one per phase), 1 weather API test.